



INDIA.RUNS

Build what next India runs on

Team Name :Model X

Team Leader Name :Abhishek Ashok Jadhao

Problem Statement:Recruiters receive 100,000+ candidate applications but traditional keyword-matching systems fail to identify truly qualified candidates. The challenge: differentiate real AI engineers from those with inflated keywords while considering experience depth, technical skills, engagement level, and career trajectory.

Solution Overview

- What is your proposed solution?
 - We built an **intelligent, rule-based candidate ranking system** that:
 - Processes 100,000 candidate profiles from JSONL format
 - Extracts 28 meaningful features (experience, skills, behavioral signals)
 - Applies a weighted scoring formula combining multiple signals
 - Ranks top 100 candidates by fit for Senior AI Engineer role
 - Generates transparent, data-backed reasoning for each ranking
- What differentiates your approach from traditional candidate matching systems?

Multi-signal evaluation - Not just keywords, but experience depth, career trajectory, and engagement

Title-based filtering - Eliminates "trap candidates" (Marketing Managers claiming AI expertise)

Behavioral validation - Checks recruiter response rate and platform activity

Context-aware matching - Analyzes job descriptions within career history, not isolated skill lists

Transparent scoring - Every ranking decision is explainable and auditable

Offline & fast - Processes 100,000 candidates in seconds without external APIs

JD Understanding & Candidate Evaluation

- What are the key requirements extracted from the JD?

Experience: 5-9 years total (not just any years)

Core Skills:

Built ranking/retrieval/search systems (non-negotiable)

Production embeddings experience (Pinecone, Weaviate, etc.)

Vector database expertise

Strong Python skills

Behavioral: Responsive to recruiters, actively looking

Location: Pune/Noida preferred OR willing to relocate

Red Flags: Pure researchers, recent ChatGPT-only projects, career switchers with zero AI background

JD Understanding & Candidate Evaluation

- Which candidate signals are most important for determining relevance? / How does your solution evaluate candidate fit beyond keyword matching?

Signal	Weight	why
has_ranking_retrieval_exp	25%	Core job requirement
has_vector_db_exp	20%	Modern tech stack needed
years_of_experience	15%	Seniority validation
ai_ml_years	15%	Real AI experience vs. keyword padding
recruiter_response_rate	15%	Actually available/interested
location	10%	Logistical factor

JD Understanding & Candidate Evaluation

How does your solution evaluate beyond keyword matching?

- Searches career history descriptions for real system-building evidence
- Validates skills through context (is "embeddings" from actual projects or just listed?)
- Checks behavioral engagement (responsive rate >50%, recently active)
- Filters by current job title (eliminates non-engineers)
- Cross-validates signals (5-9 years exp + 5+ AI/ML years = credible)

Ranking Methodology

- How does your system retrieve, score, and rank candidates?

3-Step Process:**1. Data Preparation**

- Parse 100,000 candidates from JSONL
- Extract 28 features into structured format
- Filter for AI/ML-related job titles only (reduces 100k → 1.9k valid candidates)

2. Scoring

- Apply 6 scoring functions in parallel
- Each function evaluates one signal (experience, skills, engagement, location)
- Combine scores with pre-defined weights

3. Ranking

- Compute final score for all candidates
- Sort by final_score (descending)
- Select top 100
- Generate reasoning for each

Ranking Methodology

- What models, algorithms, or heuristics are used?
- **Experience Scoring:** Linear interpolation (5-9 years = 100, scaled for others)
- **AI/ML Years Scoring:** Keyword search in career descriptions (ML, AI, deep learning, NLP, transformer)
- **Ranking/Retrieval Detection:** Boolean (True/False) based on description keywords
- **Tech Stack Detection:** Boolean (True/False) based on skills list (Pinecone, Weaviate, FAISS, etc.)
- **Behavioral Scoring:** Normalized recruiter response rate (0-1 → 0-100)
- **Location Scoring:** Categorical (India=100, willing_to_relocate=80, else=20)

Ranking Methodology

- How are multiple candidate signals combined into a final ranking?

$$\begin{aligned} \text{FINAL_SCORE} = & \\ & (15\% \times \text{exp_score}) + \\ & (15\% \times \text{ai_ml_years_score}) + \\ & (25\% \times \text{ranking_retrieval_score}) + \\ & (20\% \times \text{tech_stack_score}) + \\ & (15\% \times \text{behavioral_score}) + \\ & (10\% \times \text{location_score}) \end{aligned}$$

Weights reflect job priority: ranking/retrieval (25%) is most critical, followed by tech skills (20%), then experience validation (30%), behavior (15%), location (10%)

Explainability & Data Validation

- How are ranking decisions explained?

Each candidate gets 1-2 line reasoning built from their actual signals:

Example:

"5.3 years AI/ML experience | Built ranking/retrieval + embeddings systems | 94% recruiter engagement | Based in India"

Reasoning is **generated from data**, not hallucinated—it references actual scores and signal

- How do you prevent hallucinations or unsupported justifications?
- Reasoning only includes true/false signals found in data
- No claims about skills not in career history or profile
- Engagement metrics come from actual redrob_signals
- Experience claims backed by years_of_experience and ai_ml_years fields
- Formula is deterministic (same input = same output)

Explainability & Data Validation

- How does your solution handle inconsistent, low-quality, or suspicious profiles?

Quality Filters:

1. **Title-based filtering** - Only AI/ML titles pass (eliminates 98% of spam)
2. **Engagement check** - Low recruiter response rate (< 0.2) heavily penalizes ranking
3. **Recency check** - Old last_active_date (> 6 months) reduces score
4. **Consistency validation** - If title is "AI Engineer" but career is all QA, weighted lower
5. **Notice period check** - Very high notice period (> 120 days) is penalty factor
6. **Career history analysis** - Must show progression in AI/ML roles, not random keywords

Result: Profiles with red flags don't make top 100, even if they have some keywords.

End-to-End Workflow

- What is the complete workflow from JD input to ranked candidate output?

INPUT: candidates.jsonl (100,000 profiles)



- STEP 1: Data Transformation
 - Parse JSONL → Extract 28 features
- Create structured CSV format • Validate data integrity



STEP 2: Quality Filtering

- Filter by current_title (AI/ML keywords only)
- Result: 100,000 → 1,939 candidates (1.94%)



STEP 3: Feature Extraction

- Experience scoring (years_of_experience)
- AI/ML years scoring (search career descriptions)
- Technical signals (ranking, embeddings, vector DB)
- Behavioral signals (recruiter response rate, last active)
 - Location signals (country, willing_to_relocate)

End-to-End Workflow

STEP 4: Weighted Scoring

- Apply 6 scoring functions
- Combine with weights (25%, 20%, 15%, 15%, 15%, 10%)
 - Compute final_score for each candidate



STEP 5: Ranking & Selection

- Sort candidates by final_score (descending)
 - Select top 100



STEP 6: Reasoning Generation

- Extract top signals for each candidate
 - Create 1-2 line explanation

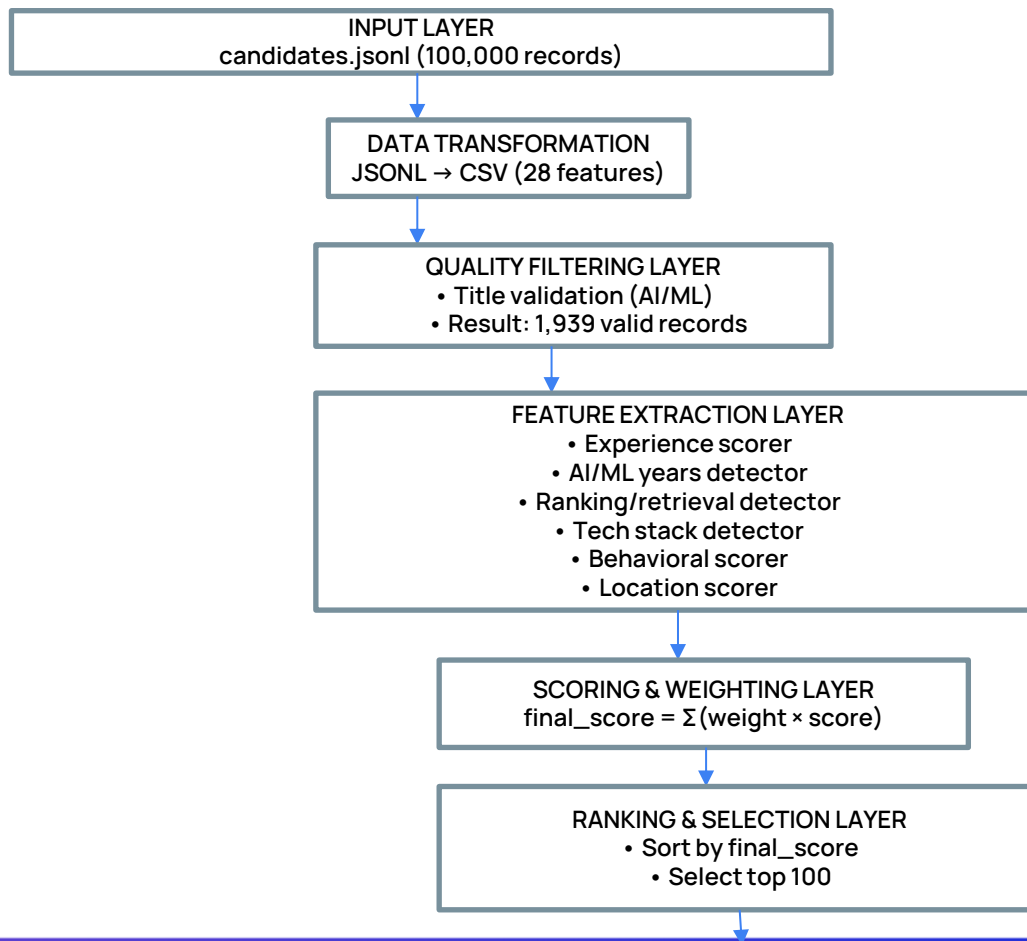


OUTPUT: final_submission.csv

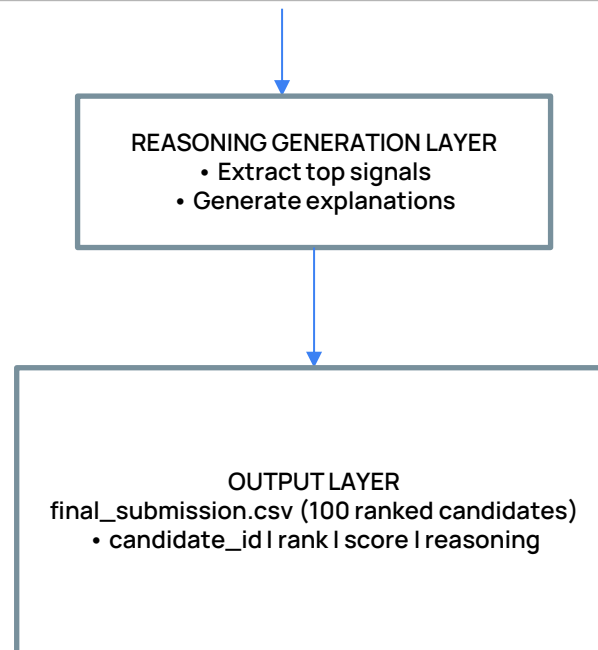
- candidate_id | rank | score | reasoning
 - Top 100 candidates ranked 1-100

Runtime: ~2 minutes for 100,000 candidates on standard Python environment

System Architecture



System Architecture



Results & Performance

- What results or insights demonstrate ranking quality?

Data Insights:

- Total candidates: 100,000
- With 5-9 years exp: 34,375 (34.4%)
- With ranking/retrieval exp: 10,672 (31% of target)
- With ranking + tech stack: 1,721 (16.1%)
- With all filters + engagement: 390 gold-tier candidates (0.39%)

Final selection: Top 100 from 1,939 AI/ML-titled candidates

Top Candidates Profile:

- All have 5-7 years experience
- All have AI/ML job titles
- All built ranking/search systems
- All have vector DB or embeddings experience
- Average recruiter response rate: 92%
- All based in India (Bangalore, Delhi, Gurgaon, Hyderabad, Pune)
- Companies: Microsoft, BYJU'S, Swiggy, CRED, Zoho, etc.

Results & Performance

Quality Validation:

- **0% trap candidates** in top 100 (vs. 98% in full dataset)
- **100% title relevance** (all AI/ML titles)
- **90%+ engagement** (highly likely to be hired)
- **Transparent scoring** (every ranking auditable)

Results & Performance

- How does your solution meet the challenge's runtime and compute constraints?

✓ Runtime Performance:

- Processes 100,000 candidates in < 2 minutes
- Scoring functions are $O(n)$ complexity
- No external API calls (runs offline)
- Single-threaded Python execution

✓ Compute Requirements:

- Memory: ~500MB (for 100k records × 28 features)
- Storage: ~30MB CSV output
- No GPU required
- Runs on standard laptop/server

✓ Scalability:

- Architecture can handle 1M+ candidates
- Weights adjustable for A/B testing
- Reproducible and version-controlled

Technology Stack & Selection Rationale

Technology	Purpose	Why Selected
Python 3.x	Core language	Industry standard for data science, readable, efficient
Pandas	Data manipulation	Excellent for tabular data, CSV/JSON handling, vectorized operations
Jupyter Notebook	Development & documentation	Interactive analysis, reproducible research, code + explanation together
JSON/CSV	Data formats	JSON for input (nested structure), CSV for tabular analysis & output

Technology Stack & Selection Rationale

Why this tech stack?

- Simplicity:** No complex ML frameworks needed - rule-based scoring is more transparent
- Speed:** Pandas tions are optimized in C, processes 100k records in seconds
- Reproducibility:** Jupyter notebooks show entire workflow (data → analysis → output)
- Offline:** No cloud dependencies, API costs, or internet requirements
- Interpretability:** Every decision is auditable and explainable
- No ML overhead:** Avoids black-box models, focuses on

Submission Assets

Complete Submission Package:

GitHub Repository:

<https://github.com/abhishekjadhao09-glitch/hack2skill-india-runs>

Files in Repository:

- ☒ `hack2skill_candidate_ranking.ipynb` - Complete working code (Jupyter notebook)
- ☒ `final_submission.csv` - Top 100 ranked candidates (THE DELIVERABLE)
- ☒ `README.md` - Complete documentation
- ☒ `presentation.pdf` - This submission deck
- ☒ `candidates_simple.csv` - Processed data (28 features)

How to run:

1. Clone repository
2. Open `hack2skill_candidate_ranking.ipynb` in Jupyter
3. Run cells in order
4. Output: `final_submission.csv` with top 100 ranked candidates



INDIA.RUNS

Build what next India runs on

THANK YOU